5.4 Functions

1. Create a function that returns a member's name using the member ID.

CODE:

```
CREATE OR REPLACE FUNCTION get_member_name (p_member_id NUMBER)
RETURN member.full_name%TYPE
IS
   v_name member.full_name%TYPE;
BEGIN
   SELECT full_name INTO v_name
   FROM member
   WHERE member_id = p_member_id;

   RETURN v_name;
END;
/

SELECT get_member_name(1003) AS member_name FROM dual;
```

OUTPUT:

```
Function created.

MEMBER_NAME
------------
Meera Thomas

1 row selected.
```

2. Create a function that returns the total number of books.

CODE:

```
CREATE OR REPLACE FUNCTION total_books
RETURN NUMBER
IS
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count FROM book;
   RETURN v_count;
END;
/

SELECT total_books() AS total_books FROM dual;
```

OUTPUT:

```
Function created.

TOTAL_BOOKS
-----------
         20

1 row selected.
```

3. Create a function that returns the number of members of a given member type.

CODE:

```
CREATE OR REPLACE FUNCTION member_count (p_type VARCHAR2)
RETURN NUMBER
IS
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count
   FROM member
   WHERE member_type = UPPER(p_type);

   RETURN v_count;
END;
/

SELECT member_count('STUDENT') AS student_members FROM dual;
```

OUTPUT:

```
Function created.

STUDENT_MEMBERS
---------------
              6

1 row selected.
```

4. Create a function that returns the number of AVAILABLE copies of a book.

CODE:

```
CREATE OR REPLACE FUNCTION available_copies (p_book_id NUMBER)
RETURN NUMBER
IS
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count
   FROM book_copy
   WHERE book_id = p_book_id
     AND availability_status = 'AVAILABLE';

   RETURN v_count;
END;
/

SELECT available_copies(4003) AS available_copies FROM dual;
```

OUTPUT:

```
Function created.

AVAILABLE_COPIES
----------------
               1

1 row selected.
```

5. Create a function that returns the total fine amount recorded for a given member ID.

CODE:

```
CREATE OR REPLACE FUNCTION total_fine (p_member_id NUMBER)
RETURN NUMBER
IS
   v_total NUMBER;
BEGIN
   SELECT NVL(SUM(amount), 0) INTO v_total
   FROM fine
   WHERE member_id = p_member_id;

   RETURN v_total;
END;
/

SELECT total_fine(1012) AS total_fine FROM dual;
```

OUTPUT:

```
Function created.

TOTAL_FINE
----------
       480

1 row selected.
```

6. Create a function that returns a staff member's salary using the staff ID.

CODE:

```
CREATE OR REPLACE FUNCTION staff_salary (p_staff_id NUMBER)
RETURN NUMBER
IS
   v_salary staff.salary%TYPE;
BEGIN
   SELECT salary INTO v_salary
   FROM staff
   WHERE staff_id = p_staff_id;

   RETURN v_salary;
END;
/

SELECT staff_salary(503) AS salary FROM dual;
```

OUTPUT:

```
Function created.

SALARY
------
 55000

1 row selected.
```

5.5 Triggers

1. Create a trigger to change a member's name to uppercase before inserting the member.

CODE:

```
CREATE OR REPLACE TRIGGER trg_member_name_upper
BEFORE INSERT ON member
FOR EACH ROW
BEGIN
   :NEW.full_name := UPPER(:NEW.full_name);
END;
/

INSERT INTO member (member_id, full_name, member_type, branch_id)
VALUES (1016, 'arjun das', 'STUDENT', 101);

SELECT member_id, full_name FROM member WHERE member_id = 1016;
```

OUTPUT:

```
Trigger created.
1 row created.

MEMBER_ID  FULL_NAME
---------  ---------
     1016  ARJUN DAS

1 row selected.
```

2. Create a trigger to stop a book's price from being set below zero.

CODE:

```
CREATE OR REPLACE TRIGGER trg_book_price_positive
BEFORE INSERT OR UPDATE OF price ON book
FOR EACH ROW
WHEN (NEW.price < 0)
BEGIN
   RAISE_APPLICATION_ERROR(-20001, 'Book price cannot be negative');
END;
/

UPDATE book SET price = -250 WHERE book_id = 4002;
```

OUTPUT:

```
Trigger created.
UPDATE book SET price = -250 WHERE book_id = 4002
       *
ERROR at line 1:
ORA-20001: Book price cannot be negative
```

3. Create a trigger to fill in the join date with today's date when it is not provided.

CODE:

```
CREATE OR REPLACE TRIGGER trg_member_join_date
BEFORE INSERT ON member
FOR EACH ROW
WHEN (NEW.join_date IS NULL)
BEGIN
   :NEW.join_date := SYSDATE;
END;
/

INSERT INTO member (member_id, full_name, member_type, branch_id)
VALUES (1017, 'KAVYA MENON', 'PUBLIC', 102);

SELECT member_id, full_name, join_date FROM member WHERE member_id = 1017;
```

OUTPUT:

```
Trigger created.
1 row created.

MEMBER_ID  FULL_NAME    JOIN_DATE
---------  -----------  ---------
     1017  KAVYA MENON  31-OCT-26

1 row selected.
```

4. Create a trigger to set a book copy's status to AVAILABLE when a loan is marked RETURNED.

CODE:

```
CREATE OR REPLACE TRIGGER trg_loan_returned
AFTER UPDATE OF loan_status ON loan
FOR EACH ROW
WHEN (NEW.loan_status = 'RETURNED')
BEGIN
   UPDATE book_copy
   SET availability_status = 'AVAILABLE'
   WHERE copy_id = :OLD.copy_id;
END;
/

UPDATE loan SET loan_status = 'RETURNED', return_date = SYSDATE
WHERE loan_id = 7001;

SELECT loan_id, loan_status FROM loan WHERE loan_id = 7001;

SELECT copy_id, availability_status FROM book_copy WHERE copy_id = 6002;
```

OUTPUT:

```
Trigger created.
1 row updated.

LOAN_ID  LOAN_STATUS
-------  -----------
   7001  RETURNED

1 row selected.

COPY_ID  AVAILABILITY_STATUS
-------  -------------------
   6002  AVAILABLE

1 row selected.
```

5. Create a trigger to stop the amount paid for a fine from being greater than the fine amount.

CODE:

```
CREATE OR REPLACE TRIGGER trg_fine_paid_limit
BEFORE INSERT OR UPDATE OF amount_paid ON fine
FOR EACH ROW
WHEN (NEW.amount_paid > NEW.amount)
BEGIN
   RAISE_APPLICATION_ERROR(-20002, 'Amount paid cannot exceed the fine amount');
END;
/

UPDATE fine SET amount_paid = amount + 100 WHERE fine_id = 9001;
```

OUTPUT:

```
Trigger created.
UPDATE fine SET amount_paid = amount + 100 WHERE fine_id = 9001
       *
ERROR at line 1:
ORA-20002: Amount paid cannot exceed the fine amount
```

6. Create a simple trigger that displays a message when a new book is added.

CODE:

```
CREATE OR REPLACE TRIGGER trg_book_added
AFTER INSERT ON book
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('New book added: ' || :NEW.title);
END;
/

INSERT INTO book (book_id, isbn, title, category, publisher_id)
VALUES (4021, '978-81-0000-021', 'Database Systems',
        'Computer Science', 201);
```

OUTPUT:

```
Trigger created.
New book added: Database Systems
1 row created.
```

5.6 Views

1. Create a view named BOOK_DETAILS showing each book's ID, title, category, publication year, and price.

CODE:

```
CREATE OR REPLACE VIEW book_details AS
SELECT book_id, title, category, publication_year, price
FROM book;

SELECT * FROM book_details;
```

OUTPUT:

```
View created.
```

BOOK_ID	TITLE	CATEGORY	PUBLICATION_YEAR	PRICE
4001	1984	Fiction	1949	399
4002	Pride and Prejudice	Fiction	1813	450
4003	Clean Code	Computer Science	2008	988.9
4004	Sapiens	History	2014	699
4005	Wings of Fire	Biography	1999	350
4006	The God of Small Things	Fiction	1997	499
4007	Introduction to Algorithms	Computer Science	2022	1099
4008	Modern Operating Systems	Computer Science	2014	1045
4009	Aarachar	Fiction	2012	525
4010	The Picture of Dorian Gray	Fiction	1890	375
4011	Designing Data-Intensive Applications	Computer Science	2017	999
4012	Learning SQL	Computer Science	2020	799
4013	Artificial Intelligence: A Modern Approach	Computer Science	2021	1318.9
4014	To Kill a Mockingbird	Fiction	1960	499
4015	India 2020	Technology	1998	325
4016	Deep Learning	Computer Science	2016	1050
4017	A Brief History of Time	Science	1988	425
4018	The Alchemist	Fiction	1988	299
4019	Machine Learning with Python	Computer Science	2022	875
4020	Qabar	Fiction	2020	280

```
20 rows selected.
```

2. Create a view named ACTIVE_MEMBERS showing members whose status is ACTIVE.

CODE:

```
CREATE OR REPLACE VIEW active_members AS
SELECT member_id, full_name, member_type, city, join_date
FROM member
WHERE status = 'ACTIVE';

SELECT * FROM active_members;
```

OUTPUT:

```
View created.

MEMBER_ID  FULL_NAME      MEMBER_TYPE  CITY            JOIN_DATE
---------  -------------  -----------  --------------  ---------
     1001  Aarav Menon    STUDENT      Kochi           10-JUN-24
     1002  Diya Nair      STUDENT      Aluva           01-JUL-24
     1003  Meera Thomas   FACULTY      Kakkanad        15-JAN-22
     1004  Rohan Joseph   PUBLIC       Kochi           11-SEP-23
     1005  Ananya Pillai  RESEARCHER   Thrippunithura  20-AUG-21
     1006  Nikhil Raj     STUDENT      Aluva           05-JUN-23
     1009  Ishita Rao     STUDENT      Kochi           10-JUN-24
     1010  Adil Khan      RESEARCHER   Kakkanad        01-OCT-19
     1011  Liya George    STUDENT      Thrippunithura  12-JUN-25
     1013  Neha Varma     FACULTY      Kakkanad        03-MAR-23
     1015  Fathima Noor   RESEARCHER   Aluva           18-APR-24

11 rows selected.
```

3. Create a view named AVAILABLE_BOOKS showing copies whose status is AVAILABLE.

CODE:

```
CREATE OR REPLACE VIEW available_books AS
SELECT copy_id, book_id, branch_id, accession_no
FROM book_copy
WHERE availability_status = 'AVAILABLE';

SELECT * FROM available_books;
```

OUTPUT:

```
View created.

COPY_ID  BOOK_ID  BRANCH_ID  ACCESSION_NO
-------  -------  ---------  ------------
   6003     4002        101  ACC-C-0002
   6005     4003        103  ACC-K-0001
   6007     4005        102  ACC-R-0002
   6008     4006        101  ACC-C-0004
   6010     4007        101  ACC-C-0005
   6011     4008        104  ACC-T-0002
   6012     4009        104  ACC-T-0003
   6015     4012        101  ACC-C-0006
   6016     4013        103  ACC-K-0004
   6018     4015        101  ACC-C-0007
   6019     4016        103  ACC-K-0005
   6020     4017        104  ACC-T-0004
   6022     4019        102  ACC-R-0005
   6023     4020        104  ACC-T-0005

14 rows selected.
```

4. Create a view showing book titles and publisher names, using the relevant tables if they exist in your database.

CODE:

```
CREATE OR REPLACE VIEW book_publishers AS
SELECT b.title, p.publisher_name
FROM book b
JOIN publisher p ON b.publisher_id = p.publisher_id;

SELECT * FROM book_publishers;
```

OUTPUT:

```
View created.

TITLE                                      PUBLISHER_NAME
-----------------------------------------  ----------------------
1984                                       Penguin Random House
Pride and Prejudice                        Penguin Random House
Clean Code                                 Pearson Education
Sapiens                                    HarperCollins
Wings of Fire                              DC Books
The God of Small Things                    Penguin Random House
Introduction to Algorithms                 MIT Press
Modern Operating Systems                   Pearson Education
Aarachar                                   DC Books
The Picture of Dorian Gray                 Oxford University Press
Designing Data-Intensive Applications      O'Reilly Media
Learning SQL                               O'Reilly Media
Artificial Intelligence: A Modern Approach Pearson Education
To Kill a Mockingbird                      HarperCollins
India 2020                                 HarperCollins
Deep Learning                              MIT Press
A Brief History of Time                    Oxford University Press
The Alchemist                              HarperCollins
Machine Learning with Python               Springer
Qabar                                      DC Books

20 rows selected.
```

5. Create a view showing each book category and the number of books in it.

CODE:

```
CREATE OR REPLACE VIEW category_summary AS
SELECT category, COUNT(*) AS book_count
FROM book
GROUP BY category;

SELECT * FROM category_summary;
```

OUTPUT:

```
View created.

CATEGORY          BOOK_COUNT
----------------  ----------
Biography                  1
Computer Science           8
Fiction                    8
History                    1
Science                    1
Technology                 1

6 rows selected.
```

6. Create a view showing staff names and their salaries.

CODE:

```
CREATE OR REPLACE VIEW staff_salaries AS
SELECT full_name, salary
FROM staff;

SELECT * FROM staff_salaries;
```

OUTPUT:

```
View created.

FULL_NAME       SALARY
-------------   ------
Lakshmi Menon    72000
Arun Das         48000
Jency Paul       55000
Rahul Dev        32000
Priya Raman      56000
Manu Suresh      34000
Elsa John        52000
Akhil Babu       30000

8 rows selected.
```